UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+

Student Name: Sahil Gupta **UID: 25MCI10266**
Branch: MCA(AI/ML) **Section/Group: 25MAM-1 A**
Semester: 2nd sem **Date of Performance: 31/03/2026**
Subject Name: TECHINCAL SKILLS **Subject Code:**

WORKSHEET 8

AIM:

To apply the concept of Stored Procedures in database operations in order to perform tasks like insertion, updating, deletion, and retrieval of data efficiently, securely, and in a reusable manner within the database system.

S/W Requirement: Oracle Database Express Edition and pgAdmin

OBJECTIVES:

Apply stored procedure concepts for database operations

Practical / Experiment Steps

Table creation:

```
CREATE TABLE Employee (
   empId INT PRIMARY KEY,
   name VARCHAR(50),
   salary INT
);

--- INSERT PROCEDURE
CREATE OR REPLACE PROCEDURE insert_employee(
   eid INT,
   ename VARCHAR,
   esalary INT
)
LANGUAGE plpgsql
AS $$
BEGIN
   INSERT INTO Employee VALUES (eid, ename, esalary);
END;
$$;

--- UPDATE PROCEDURE
CREATE OR REPLACE PROCEDURE update_salary(
   eid INT,
   new_salary INT
)
LANGUAGE plpgsql
```

```
AS $$
BEGIN
   UPDATE Employee
   SET salary = new_salary
   WHERE empId = eid;
END;
$$;
```

--- DELETE PROCEDURE

```
CREATE OR REPLACE PROCEDURE delete_employee(
   eid INT
)
LANGUAGE plpgsql
AS $$
BEGIN
   DELETE FROM Employee
   WHERE empId = eid;
END;
$$;
```

--- RETRIVE DATA USING FUNCTION

```
CREATE OR REPLACE FUNCTION get_employees()
RETURNS TABLE(empId INT, name VARCHAR, salary INT)
LANGUAGE plpgsql
AS $$
BEGIN
   RETURN QUERY
   SELECT * FROM Employee;
END;
$$;
```

--- EXECUTION PART

```
CALL insert_employee(1, 'John', 5000);
CALL update_salary(1, 6000);
CALL insert_employee(2, 'Rahul', 5000);

SELECT * FROM get_employees();

CALL delete_employee(1);
```

Data Output Messages Notifications

	empid integer	name character varying	salary integer
1	2	Rahul	5000
2	1	John	5000

Insert in the table

Data Output Messages Notifications

	empid integer	name character varying	salary integer
1	2	Rahul	5000
2	1	John	6000

update in the table

Data Output Messages Notifications

	empid integer	name character varying	salary integer
1	2	Rahul	5000

delete in the table

Outcomes:

1. **Insert Procedure** successfully adds new employee records into the Employee table.
2. **Update Procedure** modifies existing data (e.g., updating salary based on empId).
3. **Delete Procedure** removes specific employee records from the table.
4. **Function (get_employees)** retrieves and displays all employee data in a structured format.
5. Demonstrates how **procedures handle CRUD operations** while functions are used for **data retrieval in PostgreSQL**.